

# 1.2 · From LLMs to Agentic AI Systems

Week 2 · Monday Jul 20 — LLMs and Agentic AI Workflows

Generative AI & Sustainability · Munich Summer School 2026



CAL POLY



# Outline

- How LLMs work conceptually: tokens, context, and prompts
- Retrieval and tool use
- What makes an AI system agentic
- Agents, tools, memory, planning, routing, and human-in-the-loop control

# Part 1 · How LLMs actually work

No math required — three ideas: tokens, context, prompts.

# An LLM is a next-word predictor

- Trained on one task: **given text so far, predict what comes next**
- "The Isar flows through \_\_\_\_" →
  - *Munich* (probably)
- Repeat that prediction thousands of times → essays, code, plans
- Everything else — chat threads, reasoning, agents — is **built on top of this**

# Tokens: how text becomes numbers

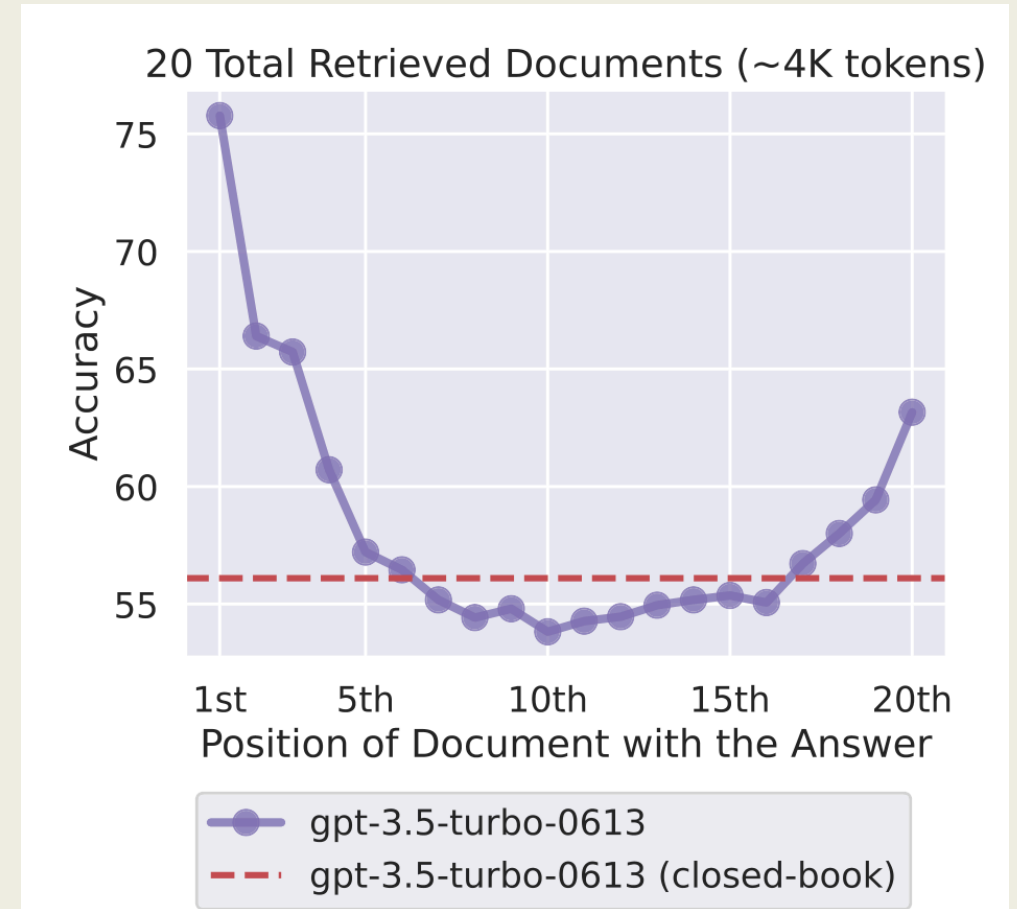
- Models don't see words — they see **tokens**: frequent chunks of characters
  - sustainability → sustain + ability
  - nachhaltigkeit → nach + haltig + keit (roughly)
  - In general German > tokens/word English
- Tokens are the **currency** of LLMs:
  - pricing is per token
  - context limits are in tokens
  - **energy scales with tokens processed**

# Context window: the model's working memory

- Everything model "sees" = **context window**:
  - prompt + convo so far + docs
- It does **not** remember yesterday's chat unless someone re-sends it
- **Working memory, not knowledge. The model's "knowledge" is frozen in its weights at training time.**

# Context window: the model's working memory

- Bigger windows are possible — but attention over long contexts costs **more compute per token**
- Position matters: models lose track of info stuck in the **middle**



# Prompts steer the prediction

- A prompt doesn't *retrieve* an answer — it **conditions** what continuation is likely
- What steers well:
  - **role and audience** — "explain to a first-year student"
  - **examples** — show one input–output pair
  - **constraints** — length, format, language
- **System prompts**: standing instructions the user never sees — every product you'll build this week has one



# In-context learning

## Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

1    Translate English to French:    ← *task description*

2    cheese => .....    ← *prompt*

# In-context learning

## One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

|   |                              |                           |
|---|------------------------------|---------------------------|
| 1 | Translate English to French: | ← <i>task description</i> |
| 2 | sea otter => loutre de mer   | ← <i>example</i>          |
| 3 | cheese =>                    | ← <i>prompt</i>           |

.....

# In-context learning

## Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:
2  sea otter => loutre de mer
3  peppermint => menthe poivrée
4  plush girafe => girafe peluche
5  cheese => .....
```

← *task description*

← *examples*

← *prompt*

# Same prompt, different answers

- Generation **samples** from the predicted distribution — it doesn't always take the top token
- *Temperature* turns randomness up or down
  - higher temp = more random output (more "creative"?)
  - Example: verbalized sampling
- Consequences for your designs:
  - answers are **not reproducible** by default
  - "it worked when we tested it" is not a guarantee
  - testing an agent means testing it **many times**

# Where the "assistant" behavior comes from

## 1 · Pre-training

Predict next token on trillions of words from the web. Learns language, facts, patterns.

## 2 · Instruction tuning

Fine-tune on question–answer examples so it responds instead of just continuing.

## 3 · Human feedback

People rank outputs; the model learns which answers humans prefer.

- **Tomorrow: what steps 1–3 cost in energy, water, and human labor.**

# Strengths and failure modes

- **Good at:** drafting, summarizing, translating, explaining, reformatting, brainstorming — fluent pattern work
- **Failure modes:**
  - **hallucination** — fluent, confident, wrong
  - facts after its training cutoff
  - precise arithmetic and logic without tools
  - knowing what it doesn't know
- Fluency  $\neq$  accuracy

# The gap an LLM can't cross alone

A plain LLM, by itself:

- **reads nothing** — no access to your files, sensors, or calendar
- **remembers nothing** — between conversations
- **does nothing** — it only writes text
- **Next, Part 2: close these three gaps**

# Part 2 · Knowing what you don't know

Retrieval brings knowledge in. Tools let actions out.



# Retrieval: look it up before answering

- **Retrieval-augmented generation (RAG)**: search *your* documents first, paste the best matches into the context, then answer
- Question → search course materials → top passages into the prompt → answer **with citations**
- Why it matters:
  - answers from **current, private** data — no retraining
  - sources can be **checked** — fights hallucination
  - a **small model + good retrieval** often beats a giant model alone

# Tool use: the model writes a request, your system acts

- The LLM outputs a **structured request**: `check_room(building="R", quiet=true)`
- **Your code** runs the tool and returns the result to the context
- The model never executes anything — it only *asks*
- Tools = external software use with **boundaries you define**:
  - calculator, calendar, room-booking API, CO<sub>2</sub> sensor readout, web search
- **You decide which tools exist, what they can touch, and what needs approval. That's system design — your job this week.**

## Worked example: "Find me a quiet place to study"

| Step          | What happens                                      |
|---------------|---------------------------------------------------|
| 1 · User asks | "Quiet spot with outlets for 2 hours?"            |
| 2 · Retrieval | Room list + booking rules from campus docs        |
| 3 · Tool call | <code>occupancy(R1.049)</code> → 12 people, 68 dB |
| 4 · Tool call | <code>occupancy(T2.015)</code> → 2 people, 41 dB  |
| 5 · Answer    | "T2.015 is quiet now — want me to hold it?"       |

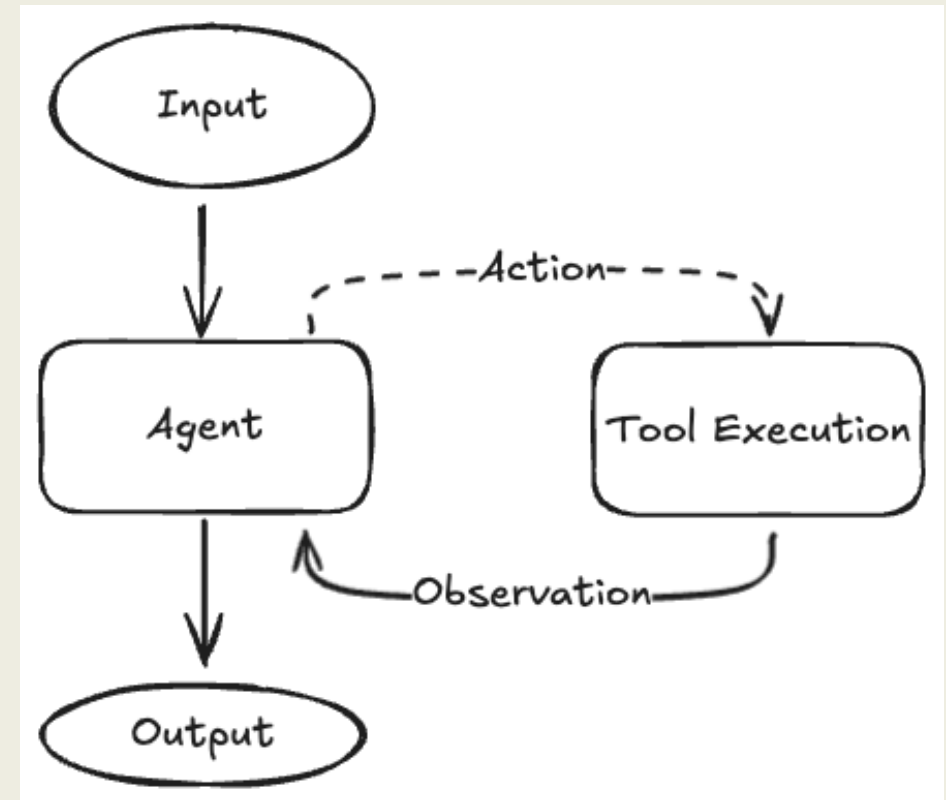
- One question → **one LLM plus two tools and a search** — already a small system.

# Part 3 · What makes a system *agentic*

A chatbot answers. An agent pursues a goal.

# The agentic loop: ReAct

- **Observe** — read the request, context, sensor data
- **Reason** — decide what's needed next
- **Act** — call a tool, fetch data, draft output
- **Repeat** — until the goal is met, then stop
  - The model **chooses its own next step** — possibly many times, no human necessary



# Building blocks: tools and memory

- **Tools** — what the agent can *do*: read sensors, book rooms, send messages
  - each tool is a capability **and** a risk surface
- **Memory** — what the agent *keeps*:
  - short-term: this conversation, this task
  - long-term: user preferences, past decisions
- Memory is where privacy lives (or dies) — **what is stored, for how long, who can see it?** (Tuesday's topic)

# Building blocks: planning and routing

- **Planning** — break a goal into steps; revise when a step fails
  - "prepare the room for a workshop" → check bookings → adjust light + air → notify the team
- **Routing** — send each step to the **cheapest thing that works**:
  - small local model → summaries, simple Q&A
  - large frontier model → only genuinely hard reasoning
- **Routing is the single biggest sustainability lever in agent design — Wednesday we measure it.**

# Building blocks: human-in-the-loop

- An **approval gate**: the agent proposes, a person decides
- Where gates are non-negotiable:
  - **irreversible** actions — deleting, sending, spending
  - decisions **about people** — grouping students, flagging behavior
  - anything a **jury** would ask "and what if it's wrong?"
- Good gates are cheap: "Book T2.015 for Thursday 14:00? [Yes / No]"



# Agents multiply inference

- One user request can trigger **dozens of model calls**: plan, then per step: reason + tool call + read result, then retry, then summarize
- Convenience for the user, **hidden multiplication** of energy and cost
- Your levers, from day one of the design:
  - fewer loop iterations, tight stop conditions
  - routing to small models
  - caching repeated answers
  - **shorter prompts and outputs** — tokens are energy

# Wrap-up

- LLMs predict tokens in a context window, steered by prompts — fluent, but they can't see, remember, or act
- Retrieval and tools close those gaps; the **agentic loop** decides its own steps toward a goal
- Six building blocks: **agents, tools, memory, planning, routing, human-in-the-loop** — every one is a design decision you'll make
- Up next: **1.3 · Agent Use Cases and Workflow Mapping** — which agents belong in your learning space, and how to draw the map